

INTRODUCING TO UVM

Author:



Table of content

- I. Testbench Architecture
- II. Connections to DUT Interfaces
- III. Analysis Components & Techniques
- IV. Sequences
- V. Register Abstraction Layer

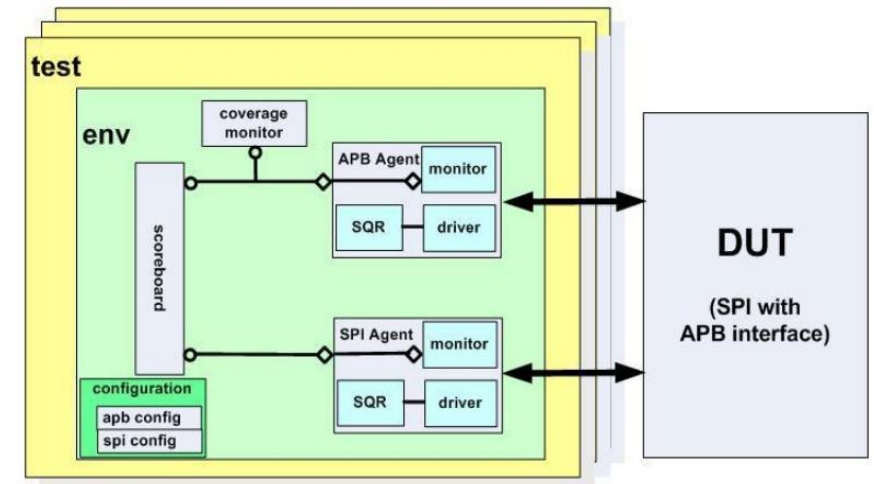
I. Testbench Architecture

1. UVM Testbench Hierarchy
2. Agent
3. Environment
4. Test
5. UVM Testbench Components
6. Phasing
7. UVM_PACKAGE

I. Testbench Architecture

1. UVM Testbench Hierarchy

- UVM Testbenches are build from classes derived from the `uvm_component` base class.
- The testbench hierarchy is dertermined by a series of 'has-a' class relationships, in other words which component contain wich other components.
- The top level class in an UVM testbench is usually known as the test class and this class is responsible for configuring the testbench.
- The UVM testbench architecture is modular to facilitate the reuse of groups of verification components either in different projects (horizontal reuse) or at a higher level of integration in the same project (vertical reuse).
- There are two main collective component types used to enable reuse
 - the env (short for environment) and the agent.

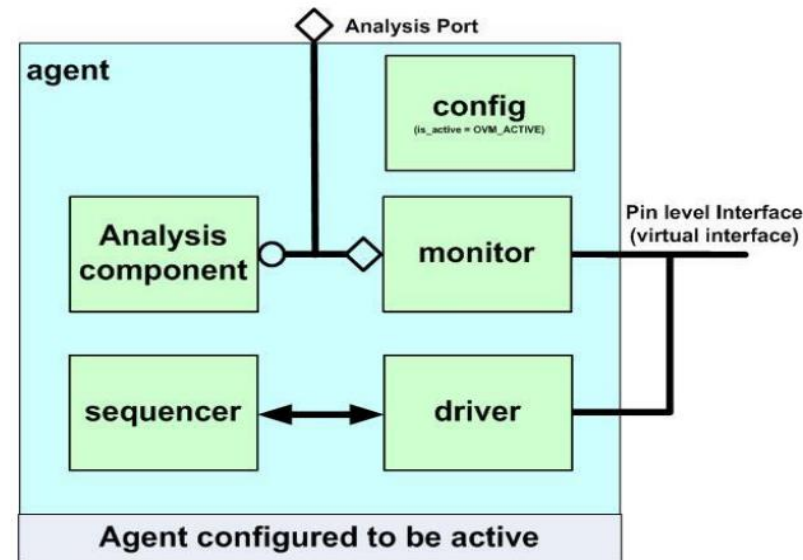


I. Testbench Architecture

2. Agent

- Purpose:

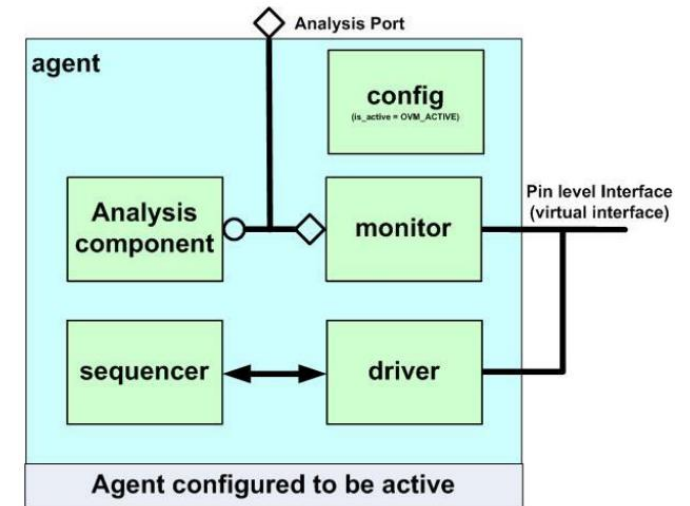
- Most DUTs have a number of different signal interfaces, each of which have their own protocol. The UVM agent collects together a group of uvm_components focused around a specific pin-level interface.
- Provide a verification component which allows users to generate and monitor pin level transactions.



I. Testbench Architecture

2. Agent

- The **contents of an agent** package will usually include:
 - A Sequence_item – also known as item, this is the **lowest level component**
 - A Sequencer - The role of the sequencer is to **route items** from a sequence where they are generated to/from a driver.
 - Driver: The driver is responsible for **converting the data** inside a series of items **into pin level transactions**.
 - A Monitor - The monitor **observes pin level activity** and **converts its observations into sequence_items** which are sent to scoreboards.
 - Configuration object - A container object, used to pass information to the agent which affects what it does and how it is built and connected.



I. Testbench Architecture

3. Environment

- Purpose:

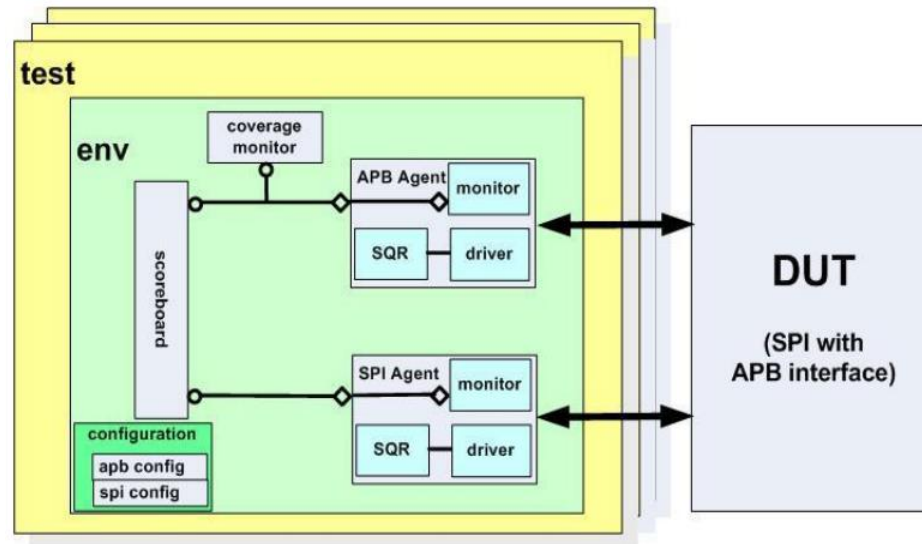
- Environment is a container component for grouping together sub-components orientated around a block, or around a collection of blocks at higher levels of integration.
- The environment is used to collect together the agents needed to communicate with the DUT's interfaces in one place.
- Like the agent, the different classes associated with the env are organized into a SystemVerilog package, which will import the agent packages.

- In addition to the agents, the env will also contain some or all of the following types of components:
 - Configuration object
 - Scoreboards
 - Functional Coverage Monitors

I. Testbench Architecture

3. Environment

- **Example:** Consider a testbench built to verify a SPI Master DUT. In this case, the UVM env has two agents – an APB agent to handle bus transfers on its APB slave port and a SPI agent to handle SPI protocol transfers on its SPI port.



I. Testbench Architecture

4. Test

- Contain environment
- The Agent may well be configured differently between test cases and so its configuration process has been split out into a separate virtual method in the base class.
- Virtual interfaces are assigned, using the *uvm_config_db::get method*.

```
class spi_test_base extends uvm_test;

// UVM Factory Registration Macro
//
`uvm_component_utils(spi_test_base)

//The environment class
spi_env m_env;
//Configuration objects
spi_env_config m_env_cfg;
apb_agent_config m_apb_cfg;
spi_agent_config m_spi_cfg;

//Method
//-----
extern virtual function void configure_apb_agent(apb_agent_config cfg);
//Standard UVM Methods:
extern function new(string name = "spi_test_base", uvm_component parent = null);
extern function void build_phase (uvm_phase phase);

endclass: spi_test_base

function spi_test_base:: new(string name = "spi_test_base", uvm_component parent = null);
|   super.new(name, parent);
endfunction

//Build the env, create the env configuration
// Including any sub configuration and assigning virtual interfaces
function void spi_test_base::build_phase(uvm_phase phase);
m_env_cfg = spi_env_config::type_id::create("m_env_cfg");
m_env_cfg.spi_rm = spi_rm;
m_apb_cfg = apb_agent_config::type_id::create("m_apb_cfg");
configure_apb_agent(m_apb_cfg);
if (!uvm_config_db #(virtual apb_if)::get(this, "", "APB_vif", m_apb_cfg.APB)) `uvm_error(...)
m_env_cfg.m_apb_agent_cfg = m_apb_cfg;
//The SPI is not configured as such
m_spi_cfg = spi_agent_config::type_id::create("m_spi_cfg");
if (!uvm_config_db #(virtual spi_if)::get(this, "", "SPI_vif", m_spi_cfg.SPI)) `uvm_error(...)
```

I. Testbench Architecture

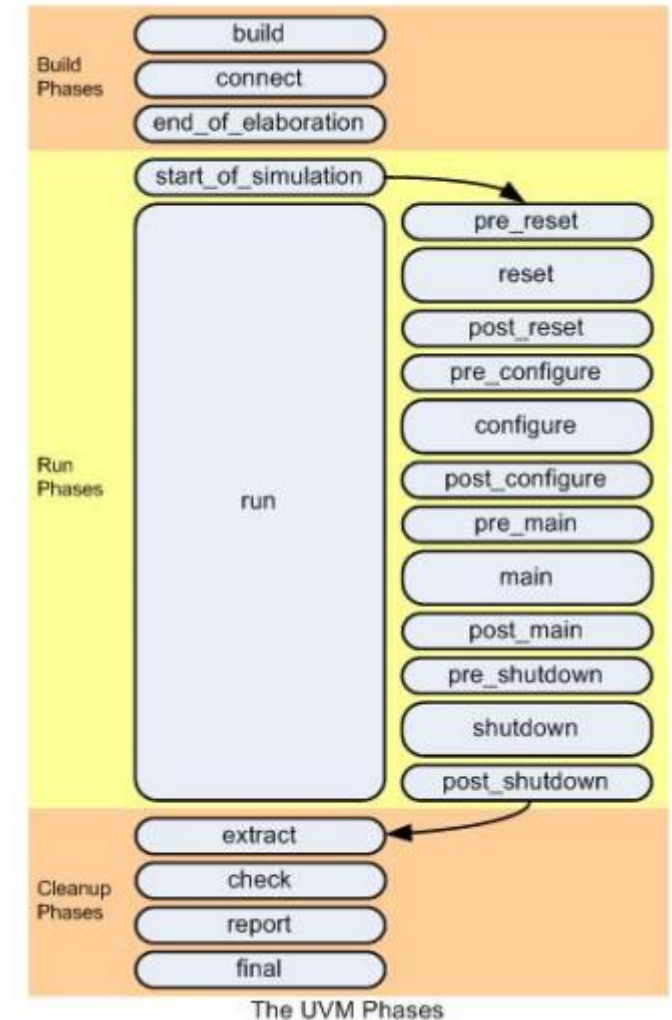
5. UVM Testbench Components

- An UVM testbench is built from component objects extended from the `uvm_component` base class. When an `uvm_component` object is created, it becomes part of the testbench hierarchy which remains in place for the duration of the simulation
- Contain interfaces, DUT, *uvm_config_db::set method*

I. Testbench Architecture

6. Phasing

- In order to have a consistent testbench execution flow, the UVM uses phases to order the major steps that take place during simulation. There are three groups of phases, which are executed in the following order:
 - Build phases - where the testbench is configured and constructed.
 - Run-time phases – where time is consumed in running the testcase on the testbench.
 - Clean up phases - where the results of the testcase are collected and reported.



I. Testbench Architecture

6. Phasing

- Build Phase:

- **Build:** Once the UVM testbench root node component is constructed, the build phase starts to execute. It constructs the testbench component hierarchy from the **top downwards**. During the build phase uvm_components are indirectly constructed using the UVM factory.
- **Connect:** The connect phase is used to make **TLM connections** between components or to assign handles to testbench resources.
- **End_of_elaboration:** The end_of_elaboration phase is used to make any **final adjustments** to the structure, configuration or connectivity of the testbench before simulation starts.

I. Testbench Architecture

6. Phasing

- **Run Phase:** The testbench stimulus is generated and executed during the run time phases which follow the build phases. After the start_of_simulation phase, the UVM executes the run phase and the phases pre_reset through to post_shutdown in parallel.
 - **Start_of_simulation:** A function which occurs **before the time consuming part** of the testbench begins. It is intended to be used for displaying banners; testbench topology; or configuration information. It is **called in bottom up order**.
 - **Run:** occurs after the start_of_simulation phase and is used for the **stimulus generation** and **checking activities** of the testbench. The run phase is implemented as a task, and all uvm_component run tasks are executed in parallel.
 - **Reset:** reserved for DUT or interface specific reset behaviour. For example, this phase would be used to **generate a reset** and to put an interface into its default state.
 - **Configure:** used to program the DUT and any memories in the testbench so that it is ready for the start of the test case. It can also be used to set signals to a state ready for the test case start.

I. Testbench Architecture

6. Phasing

- Run Phase:

- **Main:** This is where the **stimulus specified** by the test case is **generated** and **applied to the DUT**. Most data throughput will be handled by sequences started in this phase.
- **Shutdown:** used to ensure that the effects of the stimulus generated during the main phase have propagated through the DUT and that any resultant data has drained away. It might also be used to execute time consuming sequences that read status registers.
- **Other:** pre_reset, post_reset, pre_configure, post_configure, pre_main, post_main, pre_shutdown, post_shutdown (not anticipate much use)

I. Testbench Architecture

6. Phasing

- **Clean Up Phase:** used to **extract information** from scoreboards and functional coverage monitors to determine whether the test case has passed and/or reached its coverage goals. They work from the **bottom to the top** of the component hierarchy.
 - **Extract:** used to retrieve and process information from scoreboards and functional coverage monitors. This may include the calculation of statistical information used by the report phase.
 - **Check:** used to check that the DUT behaved correctly and to identify any errors that may have occurred during the execution of the testbench.
 - **Report:** used to display the results of the simulation or to write the results to file.
 - **Final:** used to complete any other outstanding actions that the testbench has not already completed.

I. Testbench Architecture

7. UVM_PACKAGE

- The UVM base class library is contained within one package called the “uvm_pkg”.

```
// Standard UVM import & include:  
import uvm_pkg::*;  
'include "uvm_macros.svh"
```

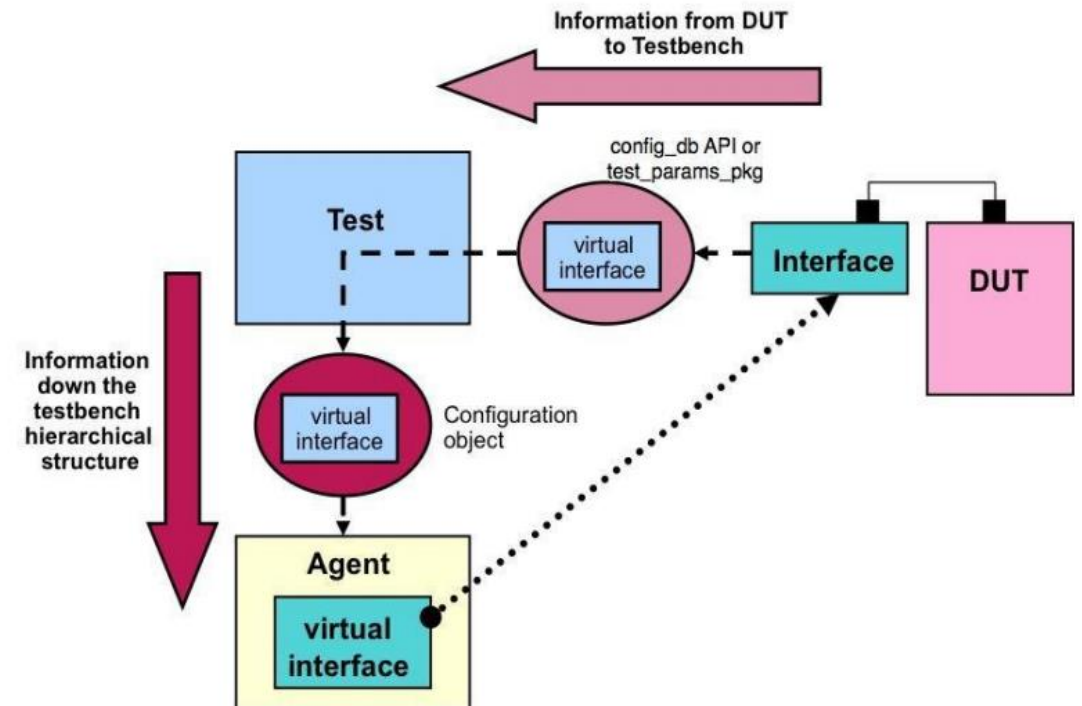

II. Connections to DUT Interfaces

1. DUT-TB Communication
2. Connect/Virtual Interface
3. Config db

II. Connections to DUT Interfaces

1. DUT-TB Communication

- The DUT and testbench belong to two different SystemVerilog instance worlds. The DUT belongs to the **static** instance world while the testbench belongs to the **dynamic** instance world.
- The DUT's ports cannot be connected directly to the testbench class objects so a different SystemVerilog means of communication, which is **virtual interfaces**, is used.
- The DUT's ports are connected to an instance of an interface.



II. Connections to DUT Interfaces

2. Connect/Virtual Interface

- **Definition:** A virtual interface is a dynamic variable that contains a reference to a static interface instance.
- Virtual interfaces are **used because a class based testbench cannot reference verilog/vhdl ports** and hence cannot directly connect to a Verilog or VHDL DUT.
- Using a virtual interface as a reference or handle to the interface instance, the testbench can access the tasks, functions, ports, and internal variables of the SystemVerilog interface.
- As the interface instance is connected to the DUT pins, the testbench can **monitor and control the DUT pins indirectly** through the interface elements.

II. Connections to DUT Interfaces

2. Connect/Virtual Interface

Example:

```
interface bus_if (input pclk);  
    logic [31:0] paddr;  
    logic [31:0] pwrdata;  
    logic [31:0] prdata;  
    logic pwrite;  
    logic psel;  
    logic penable;  
    logic presetn;  
endinterface
```

Interface

```
module tb;  
    reg clk;  
  
    always #10 clk =~ clk;  
    bus_if if (clk);  
  
    traffic u0(  
        .pclk (clk),  
        .presetn (if.presetn),  
        .paddr (if.paddr),  
        .pwrdata (if.pwrdata),  
        .psel (if.psel),  
        .pwrite (if.pwrite),  
        .penable (if.penable),  
        .prdata (if.prdata)  
    );  
  
    initial begin  
        clk <= 0;  
        uvm_config_db#(virtual bus_if)::set(null, "*", "bus_if", if);  
        run_test("");  
    end  
  
    initial begin  
        $vcdplusfile("wave_wave.vpd");  
        $vcdpluson();  
    end  
endmodule
```

Connect interface with DUT's ports in testbench and using set() method so that interface can be called in other class

II. Connections to DUT Interfaces

2. Connect/Virtual Interface

Example:

Using virtual interface and call bus_if with get() method in driver, data will be transferred to DUT through virtual interface

```
class my_driver extends uvm_driver #(bus_pkt);
    `uvm_component_utils (my_driver)

    bus_pkt pkt;

    virtual bus_if vif;

    function new (string name = "my_driver", uvm_component parent); ...
    endfunction

    virtual function void build_phase (uvm_phase phase);
        super.build_phase (phase);
        if (! uvm_config_db#(virtual bus_if)::get (this, "*", "bus_if", vif)) ...
    endfunction

    virtual task run_phase (uvm_phase phase);
        bit [31:0] data;

        vif.psel <= 0;
        vif.penable <= 0;
        vif.pwrite <= 0;
        vif.paddr <= 0;
        vif.pwdata <= 0;
        forever begin ...
        end
    endtask
```

II. Connections to DUT Interfaces

2. Connect/Virtual Interface

Example: Using virtual interface and call bus_if with get() method in monitor, data will be captured from DUT through virtual interface

```
class my_monitor extends uvm_monitor;
  `uvm_component_utils (my_monitor)
  function new (string name="my_monitor", uvm_component parent); ...
  endfunction

  uvm_analysis_port #(bus_pkt) mon_ap;
  virtual bus_if vif;

  virtual function void build_phase (uvm_phase phase);
    super.build_phase (phase);
    mon_ap = new ("mon_ap", this);
    uvm_config_db #(virtual bus_if)::get (null, "uvm_test_top.*", "bus_if", vif);
  endfunction

  virtual task run_phase (uvm_phase phase);
    fork
      forever begin
        @(posedge vif.pclk);
        if (vif.psel & vif.penable & vif.presetn) begin
          bus_pkt pkt = bus_pkt::type_id::create ("pkt");
          pkt.addr = vif.paddr;
          ...
          mon_ap.write (pkt);
        end
      end
    end
    join_none
  endtask
endclass
```

II. Connections to DUT Interfaces

3. Config db

- The `uvm_config_db` class is the recommended way to access the resource database. A resource is any piece of information that is shared between more than one component or object.
- We use `uvm_config_db::set` to put something into the database and `uvm_config_db::get` to retrieve information from the database.
- **Set method:**
 - The full signature of the set method is **`void uvm_config_db #( type T = int )::set( uvm_component cntxt , string inst_name , string field_name , T value );`**
 - T is the type of the element being configured - usually a virtual interface or a configuration object.
 - `cntxt` and `inst_name` together form a scope that is used to locate the resource within the database. The scope is formed by appending the instance name to the full hierarchical name of the context, i.e. `{cntxt.get_full_name(),".",inst_name}`.
 - `Field_name` supplies the name of the resource
 - `Value` is the thing which is actually going to be put into the database.

II. Connections to DUT Interfaces

3. Config db

- Get method:

- The full signature of the get method is **bit uvm_config_db #(type T = int)::get(uvm_component cntxt , string inst_name , string field_name , ref T value);**
 - T is the type of the element being configured - usually a virtual interface or a configuration object.
 - cntxt and inst_name together form a scope that is used to locate the resource within the database. The scope is formed by appending the instance name to the full hierarchical name of the context, i.e. {cntxt.get_full_name(),".",inst_name}.
 - Field_name supplies the name of the resource
 - Value is the thing which is going to be retrieved from the database. If the call to get is successful, this value will be overwritten.
 - The method returns 1 if it is successful and 0 if there is no such resource of this type at this location.

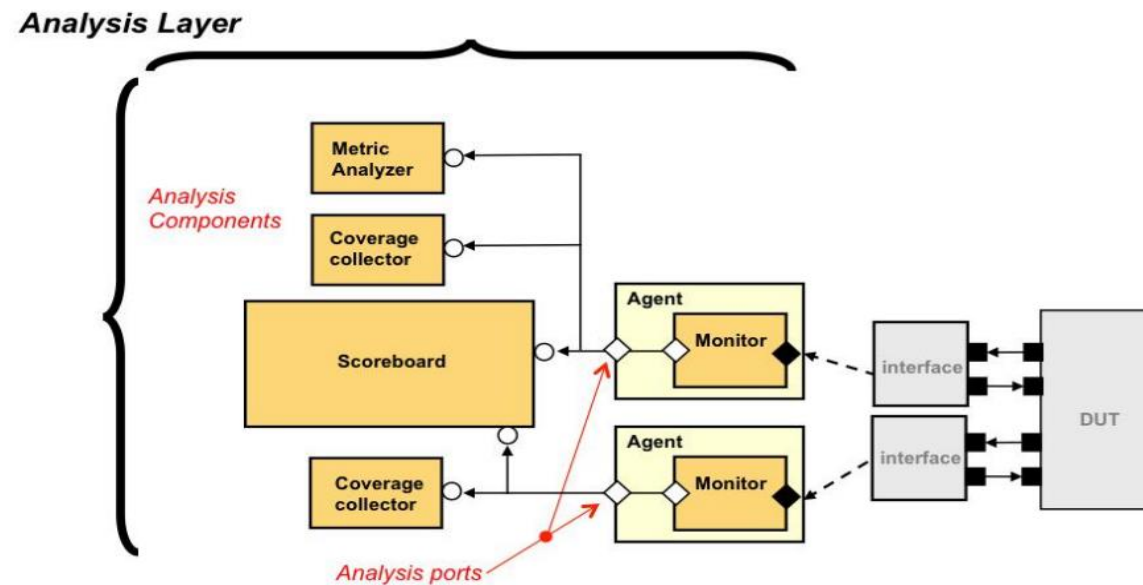
III. Analysis Components & Techniques

1. Overall
2. Analysis Port
3. Monitor
4. Scoreboard

III. Analysis Components & Techniques

1. Overall

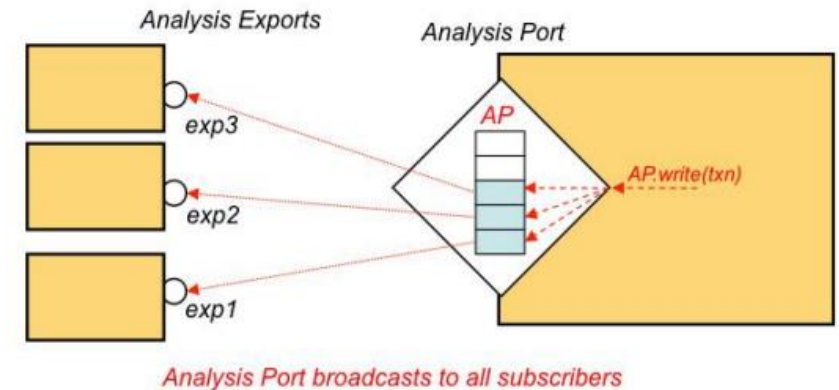
- Verifying a design consists of two major parts: **stimulus generation** and an **analysis of the designs response**.
- Stimulus generation sets up the device and puts it in a particular state, then the analysis part actually performs the verification.
- The analysis portion of a testbench is made up of components that observe behavior and make a judgment whether or not the device conforms to its specification.



III. Analysis Components & Techniques

2. Analysis Port

- Analysis ports, analysis exports and analysis fifos follow the standard UVM transaction-level communication semantics. An analysis port requires an implementation of **write()** to be connected to it.
- **Ports** provide a local object through which code can call a function. **Exports** are connection points on components that provide the implementation of the functions called through the ports. **Ports and exports are connected through a call to the `connect()` function.**
- The Analysis port provides a single void function named **write()** to perform the broadcast behavior.



III. Analysis Components & Techniques

2. Analysis Port

- Example :

Monitor send captured “reg_item” through uvm_analysis_port using *write()* function. The analysis_port is parameterized with “reg_item” type.

```
class monitor extends uvm_monitor;
`uvm_component_utils(monitor)
function new(string name="monitor", uvm_component parent=null);
    super.new(name, parent);
endfunction

uvm_analysis_port #(reg_item) mon_analysis_port;
virtual reg_if vif;
semaphore sema4;

virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db#(virtual reg_if)::get(this, "", "reg_vif", vif))
        `uvm_fatal("MON", "Could not get vif")
    sema4 = new(1);
    mon_analysis_port = new ("mon_analysis_port", this);
endfunction

virtual task run_phase(uvm_phase phase);
    super.run_phase(phase);
    // This task monitors the interface for a complete
    // transaction and writes into analysis port when complete
    forever begin
        @ (posedge vif.clk);
        if (vif.sel) begin
            reg_item item = new;
            item.addr = vif.addr;
            item.wr = vif.wr;
            item.wdata = vif.wdata;

            if (!vif.wr) begin
                @(posedge vif.clk);
                item.rdata = vif.rdata;
            end
            `uvm_info(get_type_name(), $sformatf("Monitor found packet %s", item.convert2str()), UVM_LOW)
            mon_analysis_port.write(item);
        end
    end
endtask
endclass
```

III. Analysis Components & Techniques

2. Analysis Port

- Example :

Scoreboard instantiate `uvm_analysis_imp` and implement `write()` function. Scoreboard receives item whenever monitor calling `write()`.

```
class scoreboard extends uvm_scoreboard;
  `uvm_component_utils(scoreboard)
  function new(string name="scoreboard", uvm_component parent=null);
    super.new(name, parent);
  endfunction

  reg_item refq[DEPTH];
  uvm_analysis_imp #(reg_item, scoreboard) m_analysis_imp;

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    m_analysis_imp = new("m_analysis_imp", this);
  endfunction

  virtual function write(reg_item item);
    if (item.wr) begin
      if (refq[item.addr] == null)
        refq[item.addr] = new;

      refq[item.addr] = item;
      `uvm_info(get_type_name(), $sformatf("Store addr=0x%0h wr=0x%0h data=0x%0h", item.addr, item.wr, item.wdata), UVM_LOW)
    end

    if (!item.wr) begin--
    end
  endfunction
endclass
```

III. Analysis Components & Techniques

2. Analysis Port

- Example :

Environment connect
uvm_analysis_port with uvm_analysis_imp in
connect phase using *connect()* function.

```
class env extends uvm_env;
`uvm_component_utils(env)
function new(string name="env", uvm_component parent=null);
    super.new(name, parent);
endfunction

agent      a0;      // Agent handle
scoreboard sb0;     // Scoreboard handle

virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    a0 = agent::type_id::create("a0", this);
    sb0 = scoreboard::type_id::create("sb0", this);
endfunction

virtual function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    a0.m0.mon_analysis_port.connect(sb0.m_analysis_imp);
endfunction
endclass
```

III. Analysis Components & Techniques

3. Monitor

- A monitor component is similar to a driver component in that they both perform a translation between actual signal activity and an abstract representation of that activity.
- The key difference between a Monitor and a Driver is that a Monitor is **always passive**. It **does not drive any signals on the interface**.
- Monitors should extend `uvm_monitor`. They should have one analysis port and a virtual interface handle that points to a DUT interface.
- A monitor must have knowledge of protocol in order to detect recognizable patterns in signal activity. Detection can be done by writing protocol-specific state machine code in the monitor's `run()` task.

III. Analysis Components & Techniques

3. Monitor

- **Example:** Monitor muss recognize APB protocol

```
class my_monitor extends uvm_monitor;
  `uvm_component_utils (my_monitor)
  function new (string name="my_monitor", uvm_component parent);
    super.new (name, parent);
  endfunction

  uvm_analysis_port #(bus_pkt)  mon_ap;
  virtual bus_if                vif;

  virtual function void build_phase (uvm_phase phase); ...
endfunction

virtual task run_phase (uvm_phase phase);
  fork
    forever begin
      @(posedge vif.pclk);
      if (vif.psel & vif.penable & vif.presetn) begin
        bus_pkt pkt = bus_pkt::type_id::create ("pkt");
        pkt.addr = vif.paddr;
        if (vif.pwrite) begin
          `uvm_info ("adapter", $sformatf ("mon_write"), UVM_MEDIUM)
          pkt.data = vif.pwdata;
        end
        else begin
          `uvm_info ("adapter", $sformatf ("mon_read"), UVM_MEDIUM)
          pkt.data = vif.prdata;
        end
        pkt.write = vif.pwrite;
        mon_ap.write (pkt);
      end
    end
  join_none
endtask
endclass
```

III. Analysis Components & Techniques

4. Scoreboard

- Scoreboard's determine whether or not the DUT is functioning properly. Scoreboard consists of 2 parts:
 - The first step is determining what exactly is the correct functionality.
 - Once the correct functionality is predicted, the scoreboard can then evaluate whether or not the actual results observed on the DUT match the predicted results.
 - UVM support using matlab in scoreboard.
- => The scoreboard is usually the most difficult part of the testbench to write

III. Analysis Components & Techniques

4. Scoreboard

- **Example:** scoreboard to check correctness of a decoder.
 - Whenever monitor call write() function, scoreboard receive item and perform correctness check

```
class scoreboard extends uvm_scoreboard;
  `uvm_component_utils(scoreboard)
  function new(string name="scoreboard", uvm_component parent=null);
    super.new(name, parent);
  endfunction

  bit [31:0] exp_instr;
  string type_decoded = "";

  uvm_analysis_imp #(item, scoreboard) m_analysis_imp;

  virtual function void build_phase(uvm_phase phase);--
  endfunction

  virtual function write(item DUT_item);
    if(DUT_item.rstn)begin
      // self decoding with golden model "tmp_decoded"
      decode_ctrl_sig_s tmp_decoded = instr_decode(DUT_item);

      // check the equality of input- and output-instruction--
      // check iq_type
      check_is_br(type_decoded, tmp_decoded, DUT_item);

      // check is_br
      check_iq_type(type_decoded, tmp_decoded, DUT_item);

      // check bypassable
      check_bypassable(type_decoded, tmp_decoded, DUT_item);
    end
  endfunction
endclass
```

III. Analysis Components & Techniques

4. Scoreboard

- **Example:** scoreboard to check correctness of a decoder.
 - Implement model to decode

```
virtual function decode_ctrl_sig_s instr_decode(item item);  
    // RV32I  
    if((item.instr & MASK_LUI) == EXPT_LUI) begin type_decoded="LUI"; return LUI_struct; end  
    else if((item.instr & MASK_AUIPC) == EXPT_AUIPC) begin type_decoded="AUIPC"; return AUIPC_struct; end  
    else if((item.instr & MASK_JAL) == EXPT_JAL) begin type_decoded="JAL"; return JAL_struct; end  
    else if((item.instr & MASK_JALR) == EXPT_JALR) begin type_decoded="JALR"; return JALR_struct; end  
    else if((item.instr & MASK_BEQ) == EXPT_BEQ) begin type_decoded="BEQ"; return BEQ_struct; end  
    else if((item.instr & MASK_BNE) == EXPT_BNE) begin type_decoded="BNE"; return BNE_struct; end  
    else if((item.instr & MASK_BLT) == EXPT_BLT) begin type_decoded="BLT"; return BLT_struct; end  
    else if((item.instr & MASK_BGE) == EXPT_BGE) begin type_decoded="BGE"; return BGE_struct; end
```

- Function to evaluate result

```
// @author: hungnc5  
import decode_table_package::*;  
// check is br  
// calculation  
// uop.is_br := cs.is_br <- decode.scala line 588  
// checker  
function check_is_br(string type_decoded, decode_ctrl_sig_s tmp_decoded, item DUT_item);  
    if(tmp_decoded.is_br != DUT_item.is_br) begin  
        uvm_error("SCBD", $sformatf("ERROR ! type decoded=%s out_instr=%0h out_is_br=%0h exp_is_br=%0h",  
            type_decoded, DUT_item.instr, DUT_item.is_br, tmp_decoded.is_br))  
    end  
endfunction  
  
// check iq type  
// calculation  
// uop.iq_type := cs.iq_type <- decode.scala line 529  
// checker  
function check_iq_type(string type_decoded, decode_ctrl_sig_s tmp_decoded, item DUT_item);  
    if(tmp_decoded.iq_type != DUT_item.iq_type) begin  
        uvm_error("SCBD", $sformatf("ERROR ! type decoded=%s out_instr=%0h out_iq_type=%0h exp_iq_type=%0h",  
            type_decoded, DUT_item.instr, DUT_item.iq_type, tmp_decoded.iq_type))  
    end  
endfunction
```

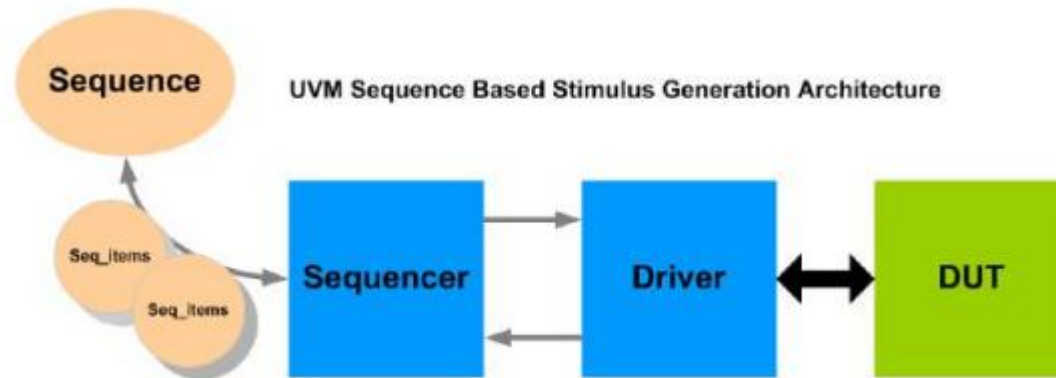
IV. Sequences

1. Introduce
2. Sequence Item
2. Constrain Item random
4. Sequence and Sequencer
5. Driver

IV. Sequences

1. Introduce

- Sequences are the primary means of generating stimulus in the UVM. The fact that sequences and sequence_items are objects means that they can be easily randomized to generate interesting stimulus.
 - In the UVM sequence architecture, sequences are responsible for the stimulus generation flow and send sequence_items to a driver via a sequencer component.
 - The driver is responsible for converting the information contained within sequence_items into pin level activity.
 - The sequencer is an intermediate component which implements communication channels and arbitration mechanisms to facilitate interactions between sequences and drivers.



IV. Sequences

2. Sequence Item

- Sequence_items are the foundation on which sequences are built
- The content of the sequence_item is closely related to the needs of the driver. The driver relies on the content of the sequence_items it receives to determine which type of pin level transaction to execute.
- Sequence_items are randomized within sequences to generate traffic data objects. Therefore, stimulus data properties should generally be declared as rand, and the sequence_item should contain any constraints required to ensure that the values generated by default are legal, or are within sensible bounds.
- As sequence_items are used for both request and response traffic and a good convention to follow is that **request properties should be rand**, and that **response properties should not be rand**. This optimises the randomization process and also **ensures that any collected response information is not corrupted by any randomization** that might take place.

IV. Sequences

2. Sequence Item

- **Example:** reg_item which contain field addr, wddata, wr, rdata
 - Rdata is a response property so it should not be rand

```
class reg_item extends uvm_sequence_item;
    rand bit [`ADDR_WIDTH-1:0]    addr;
    rand bit [`DATA_WIDTH-1:0]    wdata;
    rand bit                        wr;
    bit  [`DATA_WIDTH-1:0]         rdata;

    // Use utility macros to implement standard functions
    // like print, copy, clone, etc
    `uvm_object_utils_begin(reg_item) ...
    `uvm_object_utils_end

    virtual function string convert2str(); ...
endfunction

function new(string name = "reg_item");
    super.new(name);
endfunction
endclass
```


IV. Sequences

3. Constrain Item random

Simple expressions

```
class MyClass;
    rand bit [7:0] min, typ, max;

    // Valid expression
    constraint my_range { 0 < min;
                          typ < max;
                          typ > min;
                          max < 128; }

    // Use of multiple operators in a single expression is not allowed
    constraint my_error { 0 < min < typ < max < 128; }

    // This will set min to 16 and randomize all others
    constraint my_min { min == 16; }

    // This will set max to a random value greater than or equal to 64
    constraint my_max { max >= 64; }
endclass
```

IV. Sequences

3. Constrain Item random

Inside operator

```
constraint my_range { typ > 32;  
                    typ < 256; }  
  
// typ >= 32 and typ <= 256  
constraint new_range { typ inside {[32:256]}; }  
  
// Choose from the following values  
constraint spec_range { type inside {32, 64, 128}; }
```

Weighted distribution

```
rand bit [2:0] typ;  
constraint dist1 { typ dist { 0:=20, [1:5]:=50, 6:=40, 7:=10}; }
```

IV. Sequences

4. Sequence and sequencer

- **Sequence:**
 - A *uvm_sequence* is derived from an *uvm_sequence_item*
 - An *uvm_sequence* contains a **task method called "body"**. It is the content of the body method that **determines what the sequence does**.
 - The ***start_item()*** call is made, passing the *sequence_item* handle as an argument. This call blocks until the sequencer grants the sequence and the *sequence_item* access to the driver.
 - The ***finish_item()*** call is made, which blocks until the driver has completed its side of the transfer protocol for the item. No simulation time should be consumed between *start_item()* and *finish_item()*.
- **Sequencer:**
 - The role of the sequencer is to **route *sequence_items* from a sequence** where they are generated **to/from a driver**
 - The **connection** between a driver and a sequencer is typically **made in the *connect_phase()* method of an agent**. With the standard UVM driver and sequencer base classes, the **TLM connection between a driver and sequencer is a one to one connection** - multiple drivers are not connected to a sequencer, nor are multiple sequencers connected to a driver.

IV. Sequences

4. Sequence and sequencer

- Example:

- Randomize and create item in uvm sequence

```
class gen_item_seq extends uvm_sequence;
  `uvm_object_utils(gen_item_seq)
  function new(string name="gen_item_seq");
    super.new(name);
  endfunction

  rand int num; // Config total number of items to be sent

  constraint c1 { soft num inside {[2:5]}; }

  virtual task body();
    for (int i = 0; i < num; i++) begin
      reg_item m_item = reg_item::type_id::create("m_item");
      start_item(m_item);
      m_item.randomize();
      `uvm_info("SEQ", $sformatf("Generate new item: "), UVM_LOW)
      m_item.print();
      finish_item(m_item);
    end
    `uvm_info("SEQ", $sformatf("Done generation of %0d items", num), UVM_LOW)
  endtask
endclass
```

IV. Sequences

4. Sequence and sequencer

- Example:

- Driver using *seq_item_port* to get next item
- -> drive item to interface with correct protocol
- -> *item_done()* to prepare for next item

```
class driver extends uvm_driver #(reg_item);
  `uvm_component_utils(driver)
  function new(string name = "driver", uvm_component parent=null);
    super.new(name, parent);
  endfunction

  virtual reg_if vif;

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db#(virtual reg_if)::get(this, "", "reg_vif", vif))
      `uvm_fatal("DRV", "Could not get vif")
    endfunction

  virtual task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
      reg_item m_item;
      `uvm_info("DRV", $sformatf("Wait for item from sequencer"), UVM_LOW)
      seq_item_port.get_next_item(m_item);
      drive_item(m_item);
      seq_item_port.item_done();
    end
  endtask

  virtual task drive_item(reg_item m_item);
    vif.sel <= 1;
    vif.addr <= m_item.addr;
    vif.wr <= m_item.wr;
    vif.wdata <= m_item.wdata;
    @(posedge vif.clk);
    while (!vif.ready) begin
      `uvm_info("DRV", "Wait until ready is high", UVM_LOW)
      @(posedge vif.clk);
    end
    vif.sel <= 0;
  endtask
endclass
```

IV. Sequences

4. Sequence and sequencer

- Example:

- Agent connect `seq_item_port` of driver to sequencer's `seq_item_export`

```
class agent extends uvm_agent;
  `uvm_component_utils(agent)
  function new(string name="agent", uvm_component parent=null);
    super.new(name, parent);
  endfunction

  driver    d0;    // Driver handle
  monitor    m0;    // Monitor handle
  uvm_sequencer #(reg_item) s0;    // Sequencer Handle

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    s0 = uvm_sequencer#(reg_item)::type_id::create("s0", this);
    d0 = driver::type_id::create("d0", this);
    m0 = monitor::type_id::create("m0", this);
  endfunction

  virtual function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    d0.seq_item_port.connect(s0.seq_item_export);
  endfunction
endclass
```

IV. Sequences

4. Sequence and sequencer

- Example:

- Environment instantiate agent

```
class env extends uvm_env;
  `uvm_component_utils(env)
  function new(string name="env", uvm_component parent=null);
    super.new(name, parent);
  endfunction

  agent      a0;      // Agent handle
  scoreboard sb0;     // Scoreboard handle

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    a0 = agent::type_id::create("a0", this);
    sb0 = scoreboard::type_id::create("sb0", this);
  endfunction

  virtual function void connect_phase(uvm_phase phase); ...
  endfunction
endclass
```


IV. Sequences

4. Sequence and sequencer

- Example:

- Test randomize and create sequence “seq” in run_phase.
- “seq” will then be sent to a driver via a sequencer.

```
class test extends uvm_test;
  `uvm_component_utils(test)
  function new(string name = "test", uvm_component parent=null);
    super.new(name, parent);
  endfunction

  env e0;
  virtual reg_if vif;

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    e0 = env::type_id::create("e0", this);
    if (!uvm_config_db#(virtual reg_if)::get(this, "", "reg_vif", vif))
      `uvm_fatal("TEST", "Did not get vif")

    uvm_config_db#(virtual reg_if)::set(this, "e0.a0.*", "reg_vif", vif);
  endfunction

  virtual task run_phase(uvm_phase phase);
    gen_item seq = gen_item seq::type_id::create("seq");
    phase.raise_objection(this);
    apply_reset();

    seq.randomize() with {num inside {[20:30]}; };
    seq.start(e0.a0.s0);
    #200;
    phase.drop_objection(this);
  endtask

  virtual task apply_reset();
  endtask
endclass
```


IV. Sequences

5. Driver

- Receive items from Sequencer and send stimulus to DUT through virtual interface.
- **API:**
 - *get_next_item*: This method **blocks until a REQ sequence_item is available in the sequencers request FIFO** and then returns with a pointer to the REQ object. The *get_next_item()* call implements half of the driver-sequencer protocol handshake, and it must be followed by an *item_done()* call which completes the handshake. Making another *get_next_item()* call before issuing an *item_done()* call will result in a protocol error and driver-sequencer deadlock.
 - *try_next_item*: This is a **non-blocking variant of the *get_next_item()* method**. It will **return a null pointer if there is no REQ sequence_item available in the sequencers request FIFO**. However, if there is a REQ *sequence_item* available it will complete the first half of the driver-sequencer handshake and must be followed by an *item_done()* call to complete the handshake.
 - *item_done*: The non-blocking *item_done()* method **completes the driver-sequencer handshake** and it should be called after a *get_next_item()* or a successful *try_next_item()* call.

IV. Sequences

5. Driver

- API:

- Peek: If no REQ *sequence_item* is available in the sequencer's request FIFO, the *peek()* method will block until one is available and then return with a pointer to the REQ object, having executed the first half of the driver-sequencer handshake. Any further calls to *peek()* before a *get()* or an *item_done()* call will result in a pointer to the same REQ sequence item being returned.
- Get: The *get()* method blocks until a REQ *sequence_item* is available in the sequencer's request FIFO. Once one is available, it does a complete protocol handshake and returns with a pointer to the REQ object.
- Put: The *put()* method is non-blocking and is used to place a RSP *sequence_item* in the sequencer's response FIFO. The *put()* method can be called at any time and is not connected with the driver-sequencer request handshaking mechanism.

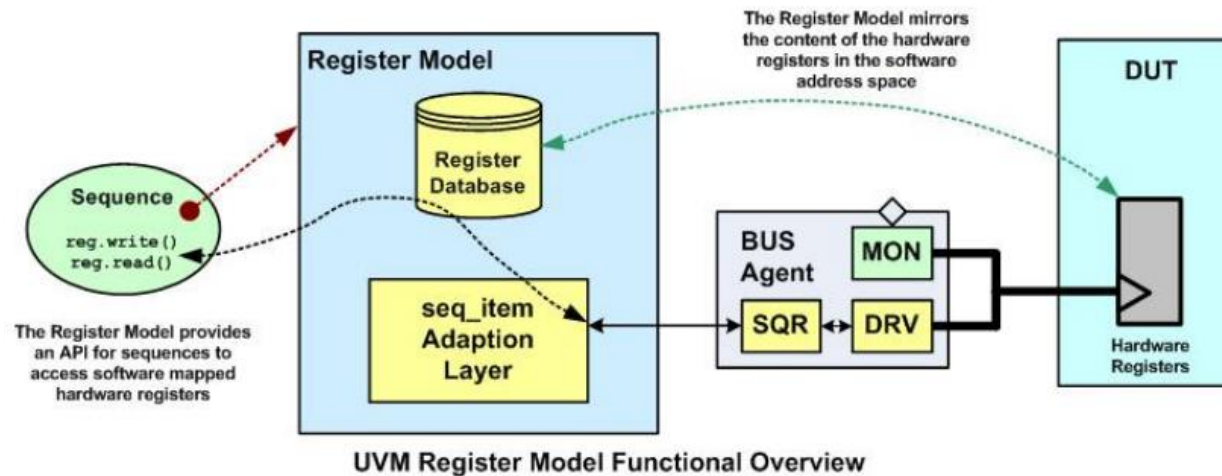
V. Register Abstraction Layer

1. Intro
2. Register Specification
3. Adapter
4. Register Predictor
5. Register Model Structure
6. Register Model Database
7. Register Model Access
7. Register Model Access
8. Library
9. Example

V. Register Abstraction Layer

1. Intro

- The UVM register model provides a way **of tracking the register content of a DUT and a convenience layer for accessing register and memory locations** within the DUT.
- The register model abstraction **reflects the structure of a hardware-software register specification**, since that is the common reference specification for hardware design and verification engineers, and it is also used by software engineers **developing firmware layer software**.
- A register model can be created using a **register generator application** or it can **be written by hand**. In both cases, the starting point is the hardware-software register specification and this is transformed into the model.



V. Register Abstraction Layer

2. Register Specification

- Hardware functional blocks connected to host processors are managed via memory mapped registers.
- In order to control and interact with the hardware, software has to read and write the registers and so the register description is organized using an abstraction which is referred to as the hardware-software interface, or as the register description.

Bit POS	31:14	13	12	11	10	9	8	7	6:0
Access	RO	R/W	R/W	R/W	R/W	R/W	R/W	RO	R/W
Name	Reserved	ACS	IE	LSB	Tx_NEG	Rx_NEG	GO_BSY	Reserved	CHAR_LEN

SPI Control Register – Reset Value = 0

V. Register Abstraction Layer

2. Register Specification

- For a given functional block, there may be multiple registers and each one will have an address which is offset from the base address of the block. To access a register, a processor will do a read or write to its address.

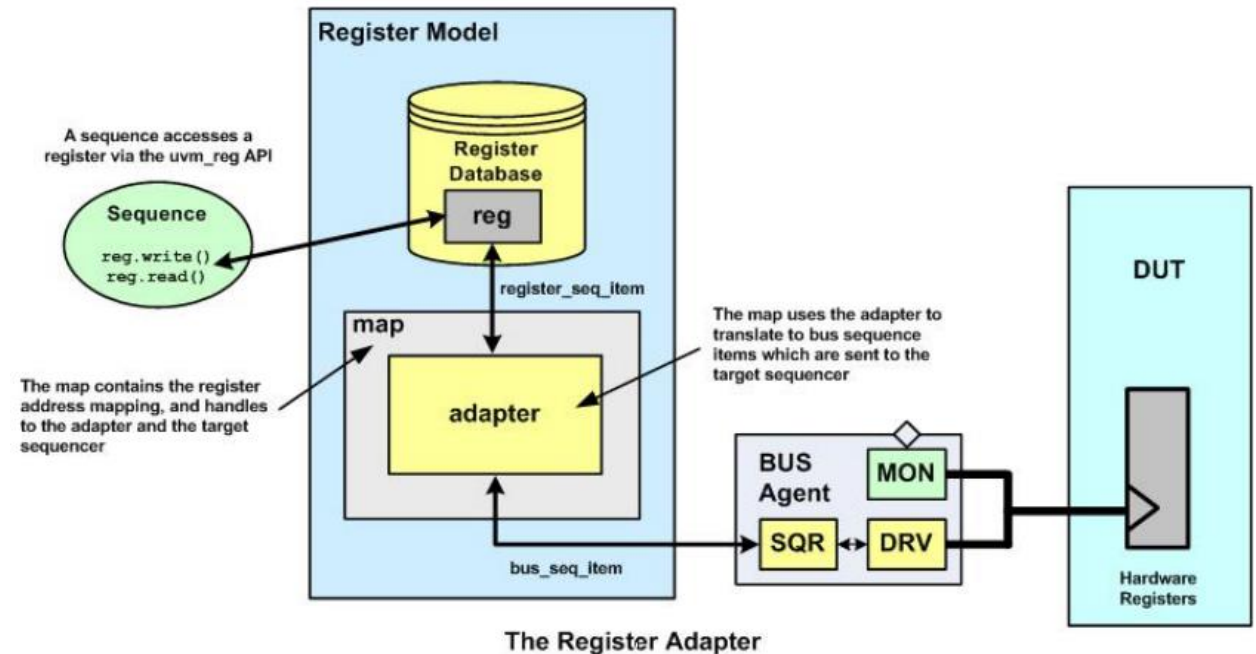
Name	Address Offset	Width	Access	Description
RX0	0x00	32	RO	Receive data register 0
TX0	0x00	32	WO	Transmit data register 0
RX1	0x04	32	RO	Receive data register 1
TX1	0x04	32	WO	Transmit data register 1
RX2	0x08	32	RO	Receive data register 2

V. Register Abstraction Layer

3. Adapter

- Purpose:

- The UVM register model **access methods generate bus read and write** cycles using generic register transactions. These transactions need to be adapted to the target bus *sequence_item*.
- The adapter needs to be **bidirectional** in order to **convert register transaction requests to bus sequence items**, and to be able to convert bus sequence item responses back to bus sequence items.
- The adapter should be implemented by extending the **uvm_reg_adapter** base class.



V. Register Abstraction Layer

3. Adapter

- Implementing an adapter:

- The **generic register item** is implemented as a struct in order to minimise the amount of memory resource it uses. The struct is defined as type **uvm_reg_bus_op** and this contains 6 fields:

Property	Type	Comment/Description
addr	uvm_reg_addr_t	Address field, defaults to 64 bits
data	uvm_reg_data_t	Read or write data, defaults to 64 bits
kind	uvm_access_e	UVM_READ or UVM_WRITE
n_bits	unsigned int	Number of bits being transferred
byte_en	uvm_reg_byte_en_t	Byte enable
status	uvm_status_e	UVM_IS_OK, UVM_IS_X, UVM_NOT_OK

V. Register Abstraction Layer

3. Adapter

- Implementing an adapter:

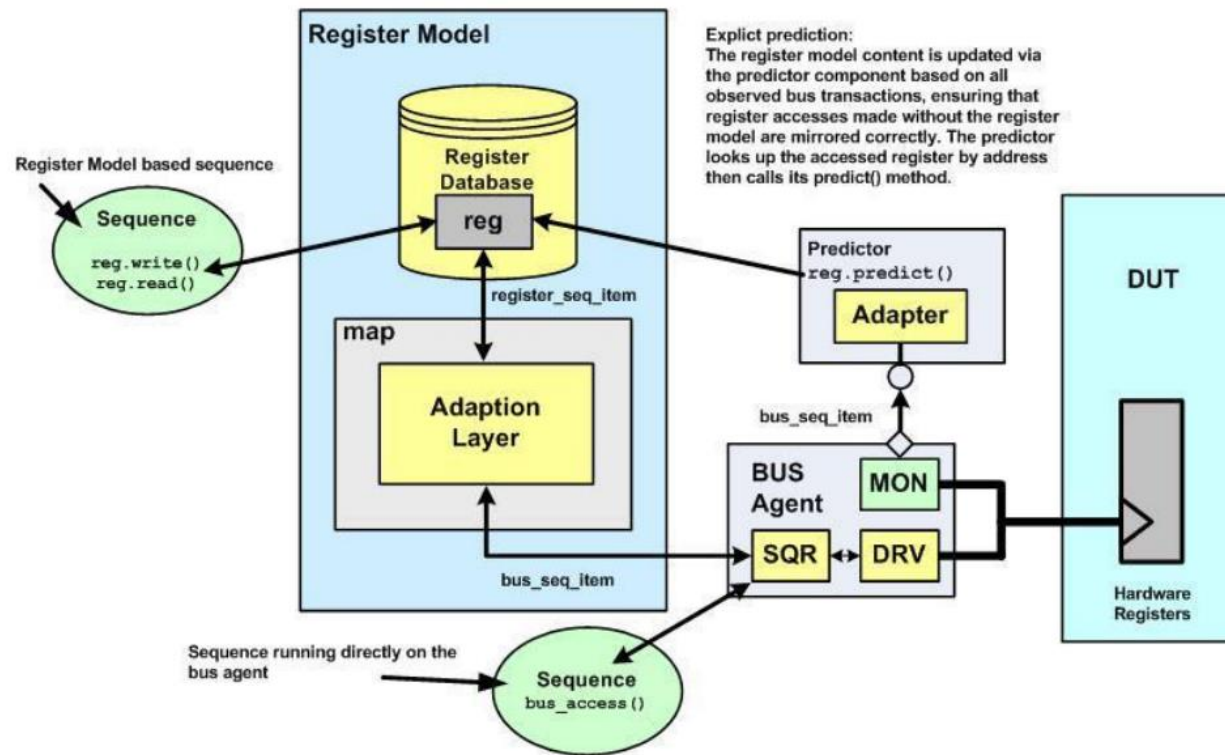
- These fields need to be mapped to/from the target bus sequence item and this is done by extending the `uvm_reg_adapter` class which contains two methods - `reg2bus()` and `bus2reg()` which need to be overlaid.

uvm_reg_adapter	
Methods	Description
Reg2bus	Overload to convert generic register access items to target bus agent sequence items
Bus2reg	Overload to convert target bus sequence items to register model items

V. Register Abstraction Layer

4. Register Predictor

- **Purpose:** update the register model to match the design contents



V. Register Abstraction Layer

4. Register Predictor

- **Implementing a register predictor:**
 - Declare register predictor with target bus transaction type
 - Build predictor in environment
 - Connect register map, adapter and analysis port to predictor

```
class reg_env extends uvm_env;
`uvm_component_utils (reg_env)
function new (string name="reg_env", uvm_component parent);
    super.new (name, parent);
endfunction

// instantiation
ral_sys_traffic      m_ral_model;          // Register Model
reg2apb_adapter      m_reg2apb;           // Convert Reg Tx <-> Bus-type packets
uvm_reg_predictor #(bus_pkt) m_apb2reg_predictor; // Map APB tx to register in model
my_agent             m_agent;             // Agent to drive/monitor transactions

// build
virtual function void build_phase (uvm_phase phase);
    super.build_phase (phase);
    m_ral_model = ral_sys_traffic::type_id::create ("m_ral_model", this);
    m_reg2apb = reg2apb_adapter :: type_id :: create ("m_reg2apb");
    m_apb2reg_predictor = uvm_reg_predictor #(bus_pkt) :: type_id :: create ("m_apb2reg_predictor", this);

    m_ral_model.build ();
    m_ral_model.lock_model ();
    uvm_config_db #(ral_sys_traffic)::set (null, "uvm_test_top", "m_ral_model", m_ral_model);
endfunction

// connect
virtual function void connect_phase (uvm_phase phase);
    super.connect_phase (phase);
    m_apb2reg_predictor.map = m_ral_model.default_map;
    m_apb2reg_predictor.adapter = m_reg2apb;
endfunction
endclass
```

V. Register Abstraction Layer

4. Register Predictor

- Predict types:

- Auto Prediction:

- In this mode, the various **access methods which cause front door accesses to take place automatically call a predict() method** using either the data that was written to the register, or the data read back from the register at the end of the bus cycle
 - This mode of operation is the **simplest to implement**, but suffers from the drawback that it **can only keep the register model up to date with the transfers that it initiates**. If any other sequences directly access the target sequencer to update register content, or if there are register accesses from other DUT interfaces, then the register model will not be updated.

V. Register Abstraction Layer

4. Register Predictor

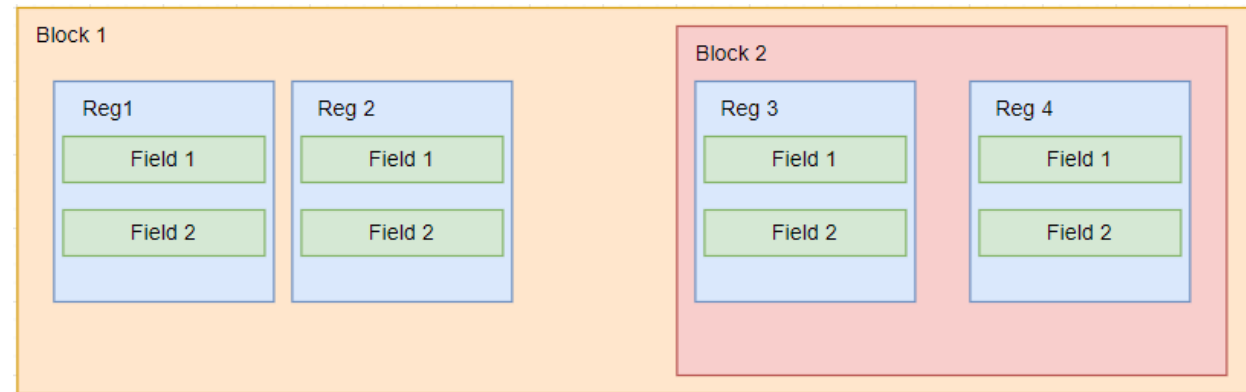
- **Predict types:**
 - **Explicit Prediction:**
 - In the explicit prediction mode of operation **an external predictor component is used to listen for target bus agent analysis transactions** and then to **call the predict() method** of the accessed register **to update its mirrored value**.
 - The predictor component **uses the adapter to convert the bus analysis transaction into a register transaction**, then uses the address field to look up the target register before calling its predict() method with the data field.
 - Explicit prediction is the default mode of prediction, to **disable** it use the **set_auto_predict() method**.
 - **Passive Prediction:** In passive prediction the register model takes no active part in any accesses to the DUT, but it is **kept up to date by the predictor when any front door register accesses take place**.

V. Register Abstraction Layer

5. Register Model Structure

- Register Fields:

- The **bottom layer is the field** which corresponds to one or more bits within a register. Each field definition is an instantiation of the **uvm_reg_field** class.
- Fields are contained within an **uvm_reg** class and they are constructed and then configured using the **configure()** method.



configure

```
function void configure(  
    uvm_reg      parent,  
    int unsigned size,  
    int unsigned lsb_pos,  
    string       access,  
    bit          volatile,  
    uvm_reg_data_t reset,  
    bit          has_reset,  
    bit          is_rand,  
    bit          individually_accessible  
)
```

V. Register Abstraction Layer

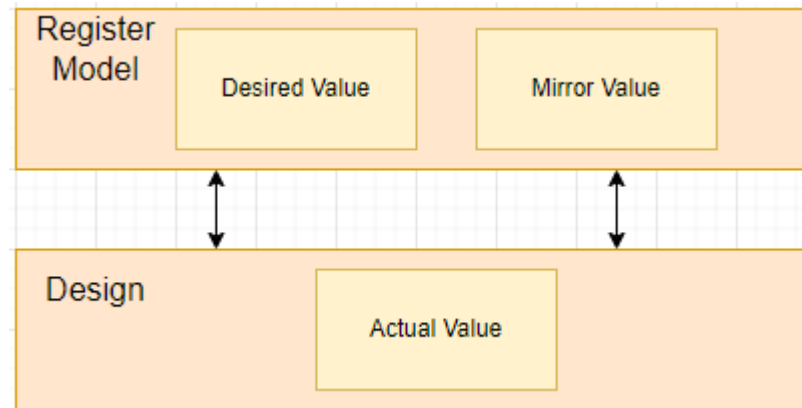
5. Register Model Structure

- **Register:**
 - Registers are **modelled by extending the uvm_reg class** which is a **container for field** objects.
 - The register class contains a **build() method** which is **used to create and configure the fields**.
- **Register Map:**
 - The map provides information on the offset of the registers, memories and/or register blocks contained within it.
 - The map is also used to identify which bus agent register based sequences will be executed on
- **Register Block:**
 - This class can be used as a **container for registers** and **memories** at the block level, representing the registers at the hardware functional block level, **or** as a container for **multiple blocks** representing the registers in a hardware sub-system or a complete SoC organised as blocks.
 - In order to define register and memory address offsets the block contains an address map object derived from uvm_reg_map.

V. Register Abstraction Layer

6. Register Model Database

- The register model has its own database which is intended to represent the state of the hardware registers. For each register there is a mirrored value and a desired value.
 - **Mirror Value:** The mirrored value represents the current known state of the hardware register. The **mirrored value is updated at the end of front bus read and write** cycles either based on the data value seen by the register model (auto-prediction) or based on bus traffic observed by a monitor and sent to **predictor** that updates the register model content
 - **Desired Value:** The desired value represents a state that the register model is going to use to update the hardware, but has not done so. In other words, the desired value allows the user to setup individual register fields before doing a write transfer.



V. Register Abstraction Layer

7. Register Model Access

- Access types:

- The UVM register model facilitates access to hardware registers in the DUT either through **front door accesses** or **back door accesses**.
- A front door access involves using a bus transfer cycle using the target bus agent, consequently it consumes time, **taking at least a clock cycle** to complete and so it models what will happen to the DUT in real life.
- A backdoor access uses the simulator database to directly access the register signals within the DUT, with write direction operations forcing the register signals to the specified value and read direction accesses returning the current value of the register signals. A backdoor access **takes zero simulation time** since it by-passes the normal bus protocol.

V. Register Abstraction Layer

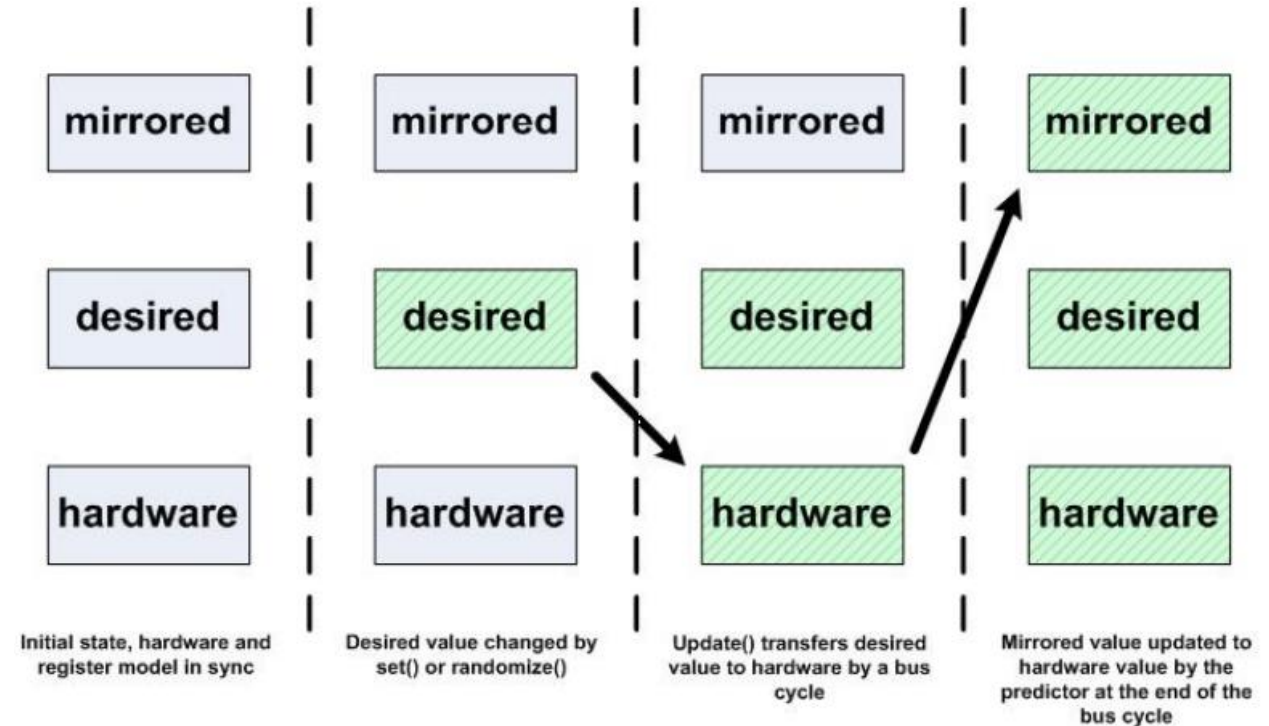
7. Register Model Access

- **Access Method:** The register model has a number of methods which can be used to read and write registers in the DUT. These methods use the desired and mirrored values to keep in step with the actual hardware register contents
 - **Read() and write()**
 - The read() method returns the value of the hardware register. When using front door accesses, calling the read() method results in a bus transfer and the desired and mirrored values of the register model are updated by the bus predictor on completion of the read cycle.
 - The write() method writes a specified value to the target hardware register. For front door accesses the mirrored and desired values are updated by the bus predictor on completion of the write cycle
 - **Get() and set()**
 - The set() and get() methods operate on the register model and **do not result in any accesses to the hardware**.
 - A call to set will assign an internal “**desired value**” for a register or field in the register model.
 - Calling get after a set will return the calculated desired value, which **may or may not be** the value you provided in the call to set. For example: For RO, the desired value is always the current mirror value, i.e. the ‘set’ has no effect whatsoever.

V. Register Abstraction Layer

7. Register Model Access

- **Get_mirrored_value()**
 - Access the current mirrored value that is stored in the register model
- **Update()**
 - If there is a difference in value between the desired and the mirrored register values, the update() method will initiate a write to a register.
 - The **mirrored value** for the register **would be set to the updated value by the predictor** at the completion of the bus write cycle.



V. Register Abstraction Layer

7. Register Model Access

- **Peek() and poke()**
 - **backdoor access** methods which can be used at the field and register level
 - **peek()** method does a **direct read** of the hardware signal state
 - the **poke()** method **forces the hardware signal state to match the data value**.
 - In both cases, the desired and mirrored values in the register model are updated automatically.
- **Randomize()**
 - Depending on whether the registers and the register fields have been defined as rand or not, they can be randomized with or without constraints
- **Mirror()**
 - The **mirror()** method **initiates a hardware read or peek access** but does not return the hardware data value.
 - A **frontdoor read** bus level operation **results in the predictor updating the mirrored value**, whereas a **backdoor peek()** automatically updates the mirrored value.

V. Register Abstraction Layer

7. Register Model Access

- **Reset()**
 - reset() method sets the register desired and mirrored values to the pre-defined register reset value
 - The reset() method is an internal register model method and does not cause any bus cycles to take place
- **Get_reset()**
 - The get_reset() method returns the pre-defined reset value for the register or the field

V. Register Abstraction Layer

7. Register Model Access

- Methode summarize

	Front door access			Back door access			
Method	Block	Register	Field	Block	Register	Field	Comment
Read()	No	Yes	Not recommend	No	Yes	Yes	
Write()	No	Yes	Not recommend	No	Yes	Yes	
Get()	No	Yes	Yes	No	Yes	Yes	Internal Method
Set()	No	Yes	Yes	No	Yes	Yes	Internal Method
Peek()	No	Yes	Yes	No	Yes	Yes	
Poke()	No	Yes	Yes	No	Yes	Yes	
Randomize()	Yes	Yes	Yes	Yes	Yes	Yes	SV Class randomize()
Update()	Yes	Yes	No	Yes	Yes	No	
Mirror()	Yes	Yes	No	Yes	Yes	No	
Reset()	Yes	Yes	Yes	Yes	Yes	Yes	Internal Method
Get_reset()	No	Yes	Yes	No	Yes	Yes	Internal Method

V. Register Abstraction Layer

8. Library

- https://verificationacademy.com/verification-methodology-reference/uvm/docs_1.2/html/index.html

9. Example

- Register specification

Name	address	WIDTH	Access
ctrl	0x0	32	RW

Bit position	3:3	2:2	1:1	0:0
Access	R/W	R/W	R/W	R/W
Field Name	profile	bl_red	bl_yellow	mod_en

V. Register Abstraction Layer

9. Example

- Register with fields

```
// Register definition for the register called "ctl"
class ral_cfg_ctl extends uvm_reg;
  rand uvm_reg_field mod_en;    // Enables the module
  rand uvm_reg_field bl_yellow; // Blinks yellow
  rand uvm_reg_field bl_red;    // Blinks red
  rand uvm_reg_field profile;   // 1 : Peak, 0 : Off-Peak

  `uvm_object_utils(ral_cfg_ctl)

function new(string name = "traffic_cfg_ctrl");
  super.new(name, 32, build_coverage(UVM_NO_COVERAGE));
endfunction: new

// Build all register field objects
virtual function void build();
  this.mod_en = uvm_reg_field::type_id::create("mod_en", get_full_name());
  this.bl_yellow = uvm_reg_field::type_id::create("bl_yellow", get_full_name());
  this.bl_red = uvm_reg_field::type_id::create("bl_red", get_full_name());
  this.profile = uvm_reg_field::type_id::create("profile", get_full_name());

  // configure(parent, size, lsb_pos, access, volatile, reset, has_reset, is_rand, individually_accessible);
  this.mod_en.configure(this, 1, 0, "RW", 0, 1'h0, 1, 0, 0);
  this.bl_yellow.configure(this, 1, 1, "RW", 0, 1'h0, 1, 0, 0);
  this.bl_red.configure(this, 1, 2, "RW", 0, 1'h0, 1, 0, 0);
  this.profile.configure(this, 1, 3, "RW", 0, 1'h0, 1, 0, 0);
endfunction
endclass

// Register definition for the register called "stat"
class ral_cfg_stat extends uvm_reg;--
endclass

// Register definition for the register called "timer"
class ral_cfg_timer extends uvm_reg;--
endclass
```


V. Register Abstraction Layer

```
// These registers are grouped together to form a register block called "cfg"
class ral_block_traffic_cfg extends uvm_reg_block;
  rand ral_cfg_ctl    ctrl;      // RW
  rand ral_cfg_timer  timer[2];  // RW
  | | | ral_cfg_stat  stat;      // RO

  `uvm_object_utils(ral_block_traffic_cfg)

  function new(string name = "traffic_cfg");
    super.new(name, build_coverage(UVM_NO_COVERAGE));
  endfunction

  virtual function void build();
    this.default_map = create_map("", 0, 4, UVM_LITTLE_ENDIAN, 0);
    this.ctrl = ral_cfg_ctl::type_id::create("ctrl", get_full_name());
    this.ctrl.configure(this, null, "");
    this.ctrl.build();
    this.default_map.add_reg(this.ctrl, `UVM_REG_ADDR_WIDTH'h0, "RW", 0);

    this.timer[0] = ral_cfg_timer::type_id::create("timer[0]", get_full_name());
    this.timer[0].configure(this, null, "");
    this.timer[0].build();
    this.default_map.add_reg(this.timer[0], `UVM_REG_ADDR_WIDTH'h4, "RW", 0);

    this.timer[1] = ral_cfg_timer::type_id::create("timer[1]", get_full_name());
    this.timer[1].configure(this, null, "");
    this.timer[1].build();
    this.default_map.add_reg(this.timer[1], `UVM_REG_ADDR_WIDTH'h8, "RW", 0);

    this.stat = ral_cfg_stat::type_id::create("stat", get_full_name());
    this.stat.configure(this, null, "");
    this.stat.build();
    this.default_map.add_reg(this.stat, `UVM_REG_ADDR_WIDTH'hc, "RO", 0);
  endfunction
endclass
```

9. Example

- Register are grouped together to form a register block

```
// The register block is placed in the top level model class definition
class ral_sys_traffic extends uvm_reg_block;
  rand ral_block_traffic_cfg cfg;

  `uvm_object_utils(ral_sys_traffic)
  function new(string name = "ral_sys_traffic");
    super.new(name);
  endfunction

  function void build();
    this.default_map = create_map("", 0, 4, UVM_LITTLE_ENDIAN, 0);
    this.cfg = ral_block_traffic_cfg::type_id::create("cfg", get_full_name());
    this.cfg.configure(this);
    this.cfg.build();
    this.default_map.add_submap(this.cfg.default_map, `UVM_REG_ADDR_WIDTH'h0);
  endfunction
endclass
```

V. Register Abstraction Layer

9. Example

- Adapter implement reg2bus() and bus2reg() function

```
class reg2apb_adapter extends uvm_reg_adapter;
  `uvm_object_utils (reg2apb_adapter)

  function new (string name = "reg2apb_adapter");
    super.new (name);
  endfunction

  virtual function uvm_sequence_item reg2bus (const ref uvm_reg_bus_op rw);
    bus_pkt pkt = bus_pkt::type_id::create ("pkt");
    pkt.write = (rw.kind == UVM_WRITE) ? 1: 0;
    pkt.addr = rw.addr;
    pkt.data = rw.data;
    `uvm_info ("adapter", $sformatf ("reg2bus addr=0x%0h data=0x%0h kind=%s", pkt.addr, pkt.data, rw.kind.name), UVM_DEBUG)
    return pkt;
  endfunction

  virtual function void bus2reg (uvm_sequence_item bus_item, ref uvm_reg_bus_op rw);
    bus_pkt pkt;
    if (! $cast (pkt, bus_item)) begin
      `uvm_fatal ("reg2apb_adapter", "Failed to cast bus_item to pkt")
    end

    rw.kind = pkt.write ? UVM_WRITE : UVM_READ;
    rw.addr = pkt.addr;
    rw.data = pkt.data;
    `uvm_info ("adapter", $sformatf ("bus2reg : addr=0x%0h data=0x%0h kind=%s status=%s", rw.addr, rw.data, rw.kind.name(), rw.status.name()), UVM_DEBUG)
  endfunction
endclass
```

V. Register Abstraction Layer

9. Example

- Environment:

- Instantiation of register model, adapter, predictor and agent
- Connect predictor's map to register model's default map
- Connect predictor's adapter to adapter

```
class reg_env extends uvm_env;
  `uvm_component_utils (reg_env)
  function new (string name="reg_env", uvm_component parent);
    super.new (name, parent);
  endfunction

  // instantiation
  ral_sys_traffic      m_ral_model;          // Register Model
  reg2apb_adapter      m_reg2apb;           // Convert Reg Tx <-> Bus-type packets
  uvm_reg_predictor #(bus_pkt) m_apb2reg_predictor; // Map APB tx to register in model
  my_agent             m_agent;             // Agent to drive/monitor transactions

  // build
  virtual function void build_phase (uvm_phase phase);
    super.build_phase (phase);
    m_ral_model      = ral_sys_traffic::type_id::create ("m_ral_model", this);
    m_reg2apb        = reg2apb_adapter :: type_id :: create ("m_reg2apb");
    m_apb2reg_predictor = uvm_reg_predictor #(bus_pkt) :: type_id :: create ("m_apb2reg_predictor", this);

    m_ral_model.build ();
    m_ral_model.lock_model ();
    uvm_config_db #(ral_sys_traffic)::set (null, "uvm_test_top", "m_ral_model", m_ral_model);
  endfunction

  // connect
  virtual function void connect_phase (uvm_phase phase);
    super.connect_phase (phase);
    m_apb2reg_predictor.map      = m_ral_model.default_map;
    m_apb2reg_predictor.adapter = m_reg2apb;
  endfunction
endclass
```

V. Register Abstraction Layer

9. Example

- Base Test: Instantiation of env

```
class base_test extends uvm_test;
  `uvm_component_utils (base_test)

  my_env      m_env;
  reset_seq   m_reset_seq;
  uvm_status_e status;

  function new (string name = "base_test", uvm_component parent);
    super.new (name, parent);
  endfunction

  // Build the testbench environment, and reset sequence
  virtual function void build_phase (uvm_phase phase);
    super.build_phase (phase);
    m_env = my_env::type_id::create ("m_env", this);
    m_reset_seq = reset_seq::type_id::create ("m_reset_seq", this);
  endfunction

  // In the reset phase, apply reset
  virtual task reset_phase (uvm_phase phase); ...
  endtask
endclass
```


V. Register Abstraction Layer

```
class reg_rw_test extends base_test;
`uvm_component_utils (reg_rw_test)
function new (string name="reg_rw_test", uvm_component parent);
    super.new (name, parent);
endfunction

// Note that main_phase comes after reset_phase, and is performed when
// DUT is out of reset. "reset_phase" is already defined in base_test
// and is always called when this test is started
virtual task main_phase(uvm_phase phase);
    ral_sys_traffic    m_ral_model;
    uvm_status_e       status;
    int                rdata;

    phase.raise_objection(this);

    m_env.m_reg_env.set_report_verbosity_level (UVM_HIGH);

    // Get register model from config db
    uvm_config_db#(ral_sys_traffic)::get(null, "uvm_test_top", "m_ral_model", m_ral_model);

    // Write 0xcafe_feed to the timer[1] register, and read it back
    `uvm_info(get_type_name(), $sformatf("initial : desired=0x%0h mirrored=0x%0h",
        m_ral_model.cfg.timer[1].get(),
        m_ral_model.cfg.timer[1].get_mirrored_value()),
        UVM_MEDIUM)
    m_ral_model.cfg.timer[1].write (status, 32'hcafe_feed);
    `uvm_info(get_type_name(), $sformatf("after write: desired=0x%0h mirrored=0x%0h",
        m_ral_model.cfg.timer[1].get(),
        m_ral_model.cfg.timer[1].get_mirrored_value()),
        UVM_MEDIUM)
    m_ral_model.cfg.timer[1].read (status, rdata);
    `uvm_info(get_type_name(), $sformatf("after read: desired=0x%0h mirrored=0x%0h",
        m_ral_model.cfg.timer[1].get(),
        m_ral_model.cfg.timer[1].get_mirrored_value()),
        UVM_MEDIUM)
```

9. Example

- Read-write test, which extends base_test and perform register check using register model

```
// Set 0xface as the desired value for timer[1] register
m_ral_model.cfg.timer[1].set(32'hface);
// `uvm_info(get_type_name(), $sformatf("after set desired=0x%0h mirrored=0x%0h", m_ral_model.cfg.
m_ral_model.cfg.timer[1].update(status);
// `uvm_info(get_type_name(), $sformatf("after update desired=0x%0h mirrored=0x%0h", m_ral_model.c
m_ral_model.cfg.timer[1].mirror(status, UVM_CHECK);
// `uvm_info(get_type_name(), $sformatf("after mirror desired=0x%0h mirrored=0x%0h", m_ral_model.c

// Predict that current value of timer[1] is 0xcafe_feed and check it is true
m_ral_model.cfg.timer[1].predict(32'hcafe_feed);
// `uvm_info(get_type_name(), $sformatf("after predict desired=0x%0h mirrored=0x%0h", m_ral_model.
m_ral_model.cfg.timer[1].mirror(status, UVM_CHECK);
// `uvm_info(get_type_name(), $sformatf("after mirror desired=0x%0h mirrored=0x%0h", m_ral_model.c

// Set desired value of the field "bl_yellow" in register ctrl to 1
// Then start bus transactions by calling "update" to update DUT with
// desired value
m_ral_model.cfg.ctrl.bl_yellow.set(1);
m_ral_model.cfg.update(status);

// Attempt to write into a R0 register "stat" with some value
m_ral_model.cfg.stat.write(status, 32'h12345678);
phase.drop_objection(this);
endtask
```